



EIP Student Guide
Technical Track
Tech 4
2026/2027



Introduction

This track is about exploring and integrating advanced technologies, solving hard technical problems, and producing a project that stands up without the team that built it. It asks you to go further than making code work: to identify what is genuinely hard in your project, to compare approaches by measuring them rather than by arguing about them, to decide, and to prove what you claim.

What the track asks of you

Two mandatory objectives, required of every team:

- **Track what's out there and use it to crack your project's hard problem:** identify what your project cannot do yet, set up a properly organized way of keeping up on that subject, experiment, decide on the basis of measurements, and actually integrate the result.
- **Structure, document and make your technical architecture reliable:** an architecture you can defend, a pipeline that checks the code on every change, tests that catch real regressions, a deployment someone else can reproduce, and a written trace of your decisions.

You then choose two complementary objectives out of four, together with your mentor:

- **A - Protect and make the most of your technology**
- **B - Open your project to outside contributions**
- **C - Work with technical experts**
- **D - Measure, test and optimize technical performance**

That choice is settled at your third mentoring session, in October, and is not changed afterwards. It commits you for the whole year.



How you are supported

You meet your mentor for 45-minute sessions, every three weeks.

Send your deliverables 48 hours before each session. The format is short: it assumes the time is spent being challenged, not discovering documents. Your mentor reads your work beforehand and arrives with questions and objections already formed.

Your mentor writes up each session on the platform within 72 hours. That write-up is the record of your progress across the year, and it is what shows the jury that the work was spread out rather than produced in a rush.

Your mentor accompanies your process, not the building of your product. Your progress on development, features, technical quality and internal organization is followed separately by your Epitech pedagogical supervisor. In a mentoring session, the question is not where you are on your features, it is where you are on your track objectives.

What is at stake at the Greenlight jury

The Greenlight jury decides whether your project can go on to its final delivery in fifth year, or whether it needs to be substantially redirected.

What the jury assesses is not intention but real progress: an objective with no evidence counts as not validated. A team that fell short of its objective but can show what it did, say where it stands and explain what was missing will be judged better than a team that claims success without evidence.



The 2 mandatory objectives

1 – Track what's out there and use it to crack your project's hard problem

This objective asks you to take up a stance of technical exploration and put it at the service of a real problem. Keeping up with your field is not a documentation exercise: it exists to find out how to solve what you cannot do yet.

The hard problem

Every project has one part harder than the rest: a computation that has to fit in a given time, a synchronization, a volume of data, a matching algorithm, a hardware constraint, a level of reliability to guarantee, a running cost to keep under control.

That is what your research, your experiments and your integration have to be about.

Characterize the hard problem

- A statement of the technical problem: what your project has to do and cannot do properly yet.
- What makes it hard: a constraint of time, volume, resources, precision, reliability, cost, or compatibility.
- What fails with a naive approach: what the obvious solution gives you, and why it is not enough. With a measurement to back it up, however rough.
- Success criteria in figures: above which threshold the problem counts as solved (a time, a rate, a capacity, a cost).

Set up how you keep up

- Sources followed, listed and justified: official documentation, scientific publications, engineering blogs, repositories and releases you track, newsletters, conferences, chat servers, specialist forums. For each one, what it brings and why you kept it.
- Tools and automations in place: a feed aggregator, alerts on repositories or topics, subscriptions, a team channel fed automatically, a shared note base. The setup has to be described and demonstrable, not just asserted.
- A rhythm defined and held: how often each member spends time on this, and how much.
- Split within the team: who follows which area. Research run by one person alone does not spread.
- A research log fed through the year: date, source, subject, what it changes or could change for the project, what you did about it or why you did nothing. An entry with no conclusion does not count.
- At least 20 entries over the year, spread out over time.



Share it and use it internally

- A sharing ritual in place from November: a dedicated slot in your team meeting, a dedicated channel, a periodic note. Format is up to you, monthly at minimum.
- At least 6 internal presentations over the year, recorded: who presented what, to whom, what was taken from it.
- At least 3 things from your research actually used in the project: a library adopted, a method taken up, a fix applied, a configuration reused from a repository, a practice integrated. For each, the original source and what it changed.

Establish the state of the art on the hard problem

- A list of the existing approaches to the problem you identified: libraries, algorithms, architectures, services, published methods.
- A comparison of at least 3 approaches on explicit criteria that matter for your project: performance, cost, maturity, integration complexity, dependencies, license, community, maintenance.
- Sources cited and dated. A comparison produced without opening the original documentation or running anything shows immediately.
- Working hypotheses: which approach you think is best at this stage, and why.

Experiment: 2 prototypes of different kinds

The decision prototype, putting two approaches head to head to settle a choice:

- Two candidate solutions implemented on the same use case, with the same data set or the same scenario.
- The same protocol for both: same conditions, same measurements, same environment. That is what makes the comparison valid.
- Measurements taken and presented side by side.
- The code of both prototypes kept and accessible.

The exploration prototype, testing a lead you found, with no decision attached:

- A technology, method or architecture spotted during the year that the team does not master.
- A hypothesis stated beforehand: what you are trying to find out.
- The result obtained, including a negative one. A lead explored then dropped, with the reason, is a valid result and often more instructive than a success.



Decide

- A reasoned technical decision on how to handle the hard problem, based on the prototype measurements and not on a preference.
- The approaches you set aside, and the reason for each.
- The consequences you accept: what the chosen solution costs you (a dependency, complexity, a running cost, debt you take on).

Integrate for real

- The chosen solution integrated into the project, in production or on the main branch.
- Proof that it works: an identifiable commit or merge, a demonstration, tests covering the module concerned.
- A before and after measurement on the success criteria you set in October: is the threshold reached, and by how much you fall short if not.
- What the integration replaced, or made possible that could not be done before.
- Architecture and documentation updated to reflect the change.
- Limits acknowledged: what is not solved, and under what conditions the solution would not hold.

Take part in technical communities

- Active participation in at least 2 technical communities related to your project's subjects: chat servers, specialist forums, repositories, mailing lists, professional groups.
- At least 5 public contributions recorded: a question asked with context and a reproduction, an answer given to someone else, a documented bug report, an experience shared, taking part in a technical discussion.
- At least one external contribution, however modest: a documentation fix, a detailed report, a proposed change, an answer that was accepted.
- For each contribution: the link, the date, the context, and what came out of it.

Timeline and deliverables

Item	Due	Deliverable
Characterizing the hard problem	Before end of October	Characterization note with success criteria in figures, validated by your mentor
Research setup	Described before end of October, held through May	Description of sources, tools, rhythm and split
Research log	Fed October to May, 20 entries minimum	Complete dated log
Internal sharing	Ritual in place from November, 6 presentations over the year	Records of the presentations, list of what was reused
State of the art	Before end of December	Comparison note on 3 approaches, sources cited and dated
Decision prototype	Before end of January	Code of both candidates, protocol, measurements side by side
Exploration prototype	Between February and April	Code, hypothesis stated beforehand, result including a negative one
Technical decision	Before end of February	Decision note, approaches set aside, consequences accepted
Integration	Before end of April	Commit or merge, demonstration, before / after measurement
Technical communities	November to May	Links and screenshots of the contributions, external contribution

What you submit

- Characterization note on the hard problem, dated October, with the success criteria
- Description of your research setup: sources, tools, automations, rhythm, split
- Complete research log
- Records of the internal presentations
- State-of-the-art note with the comparison of approaches and sources cited
- Code and protocol of both prototypes, with the measurements taken
- Technical decision note
- Proof of integration: commit or merge, demonstration, tests
- Before and after measurements on the success criteria
- Links and screenshots of your contributions in the communities

2 – Structure, document and make your technical architecture reliable

This objective asks you to produce a project that stands up without you: an architecture you can defend, a pipeline that checks the code on every change, tests that actually catch regressions, a deployment someone else can reproduce, and a written trace of the decisions you made.

Design and document your architecture

- An architecture diagram showing: the components and services, what each is responsible for, the dependencies between them, the data flows and their direction, the external dependencies, and the deployment boundaries.
- A data diagram: main entities, relationships, expected volumes.
- Points of failure identified: what happens if a component goes down, if an external service stops responding, if the load doubles.
- Justification of your structuring choices, and for each one, the alternative you considered and set aside, with the reason.
- The diagram is kept up to date. A diagram that no longer matches the code in May is worth nothing.

Keep a record of technical decisions (ADR: Architecture Decision Record)

One entry per structuring decision, written at the moment it is made, in a consistent format:

- Date
- Context and problem
- Options considered
- Option chosen
- Reason
- Consequences accepted

What it is for: it carries the trace of the reasoning, not just the result.

- At least 4 decisions recorded over the year.
- They are about things that commit the project: choice of technology, how components are split, data model, exchange format, authentication strategy, deployment mode, concurrency handling, dropping a component.
- At least 2 options genuinely weighed per decision.
- Consequences spelled out: what the decision costs in dependency, complexity, performance, or debt taken on.
- At least one decision you went back on during the year, with what changed your mind.



Produce your technical documentation

- README: what the project is for, prerequisites, installation, configuration, environment variables, main commands, folder structure, how to run the tests.
- API documentation: endpoints exposed, expected parameters, response formats, error codes, example calls.
- Operations documentation: where the logs are, how to restart, what to do when something common breaks.

Set up a continuous integration pipeline

- Automatic execution on every commit or merge request: static analysis, formatting, tests, build.
- A browsable history over the whole period, showing both successful and failed runs.
- Merges blocked on failure on the main branch.
- At least one documented case where the pipeline stopped a broken change before it reached the main branch.

Measure your code quality

- Analysis tools configured and run in the pipeline: linting, formatting, static analysis, type checking where your language allows it.
- A baseline and a final measurement: number of warnings and errors at the start, at the end, and what you fixed in between.
- The rules you adopted, and the ones you deliberately disabled, with the reason for each exception.
- Technical debt logged: what you know you should fix, why you haven't, and roughly what it would cost.

Test

- The critical path identified: the flows or functions whose failure makes the product unusable.
- Automated tests on that critical path, run in the pipeline.
- Coverage figure recorded and presented, with the share corresponding to the critical path.
- At least one integration test covering a full flow, not an isolated function.
- At least 3 regressions caught by your tests, documented: what broke, which test caught it, when. A test that has never caught anything proves nothing; a test that stopped a broken release proves everything.
- Behavior on error: the failure cases you anticipated, what the system does in each, what gets logged.



Review each other's code

- No merge to the main branch without review by another team member.
- Evidence of the review: comments, change requests, discussion. Approving without a single comment is not a review.
- At least 15 merge requests reviewed over the year, spread across the team.
- At least 3 cases where a review led to a change before merging, presented.

Make your deployment reproducible

- The project must be installable and runnable by someone outside the team.
- An automated procedure where possible: containerization, install script, composition file.
- A reproducibility test carried out and dated, by someone outside the team. Record how long it took, where they got stuck, and what you had to fix in the documentation as a result.
- Configuration kept out of the code: environment variables, no secrets in the repository.

Timeline and deliverables

Item	Due	Deliverable
Architecture	First version before end of November, updated at every structuring change	Architecture diagram and data diagram, up to date
Decision record	Fed October to May, 4 decisions minimum	Complete dated record
Technical documentation	First version before end of December, corrected after the reproducibility test	README, API and operations documentation
Pipeline	Running before end of December	Link to the run history
Code quality	Baseline in December, final measurement in May	Initial and final static analysis reports, debt log
Tests	Critical path covered before end of March	Coverage report, list of regressions caught
Code review	Throughout the year	Merge request history and review comments
Reproducibility test	Before end of March	Dated report: who, how long, blockers, fixes applied



What you submit

- Architecture diagram, up to date, and data diagram
- Complete dated record of technical decisions
- README, API documentation and operations documentation
- Link to the pipeline run history
- Initial and final static analysis reports
- Test coverage report
- List of regressions caught, with dates and the tests concerned
- Merge request history and review comments
- Reproducibility test report, with who, when, how long and the blockers
- Technical debt log



The complementary objectives (you choose 2 with your mentor)

A – Protect and make the most of your technology

This objective asks you to identify what genuinely has value in what you built, to protect it proportionately, and to be able to explain it to someone else. A developer who cannot say what is hard or original about their own work can neither defend it, nor negotiate it, nor make anything of it in front of a recruiter or a partner.

Map what has value

- A list of the elements that carry value: an algorithm, a processing method, a trained model, a data set you built, a particular architecture, integration know-how, an interface you expose.
- For each: what makes it hard to reproduce, how much time it cost, what would happen if it were copied.
- An explicit distinction between what is genuinely original and what is assembly of existing building blocks. A team that is clear-eyed on this is more credible than a team that claims everything.
- Ranked by criticality: what would be damaging to disclose, what is indifferent, what would gain from being opened.

Map what is exposed

- An inventory of your exposure surfaces: public or private repository, accessible interfaces, entry points, third-party dependencies, stored data, secrets and keys.
- For each surface: who can reach it today, and what they could do with it.
- A check of your repository history: secrets, keys or personal data present in earlier commits, even if removed since.



Put proportionate measures in place

Every measure you claim has to be demonstrable in the code or the configuration, not just described.

Technical measures

- At least 3 measures actually implemented, chosen from your mapping: secrets kept out of the repository, encryption of sensitive data, access control and partitioning, rate limiting, access logging, anonymization, separation of environments.
- Proof of implementation for each: a commit, a configuration file, a screenshot of it running.
- A security analysis carried out: a security-oriented static analysis tool, a dependency audit, a documented manual review, a penetration test where relevant. Results recorded, vulnerabilities identified, what was fixed, what was not and why.

Legal measures, where they are relevant

- A timestamped filing or proof of anteriority: a sealed envelope, a digital deposit, a timestamp. The choice is justified (or the absence of any filing is justified too), which is a perfectly defensible position.
- A license applied to the code, consistent with the strategy you chose.
- A confidentiality agreement with a partner or an external tester, where a third party gets access to sensitive elements.

Document your strategy

A one-page note bringing together: what is protected, why, by what means, and what you deliberately left open.

Know how to present your technology

Technical work that cannot be told cannot be valued. This is what plays out in an interview, in front of a partner, or in front of a jury.

- A two-page technical sheet for an outside reader: the problem addressed, the approach chosen, what sets it apart from an obvious solution, the results obtained with figures, the known limits.
- A demonstration script: what you show, in what order, in how long, and what the viewer should have understood by the end.



Timeline and deliverables

Item	Due	Deliverable
Mapping the value	Before end of January	List ranked by criticality, with the original / assembly distinction
Mapping the exposure	Before end of January	Inventory of surfaces, check of the repository history
Technical measures	Before end of April	3 measures implemented, with commits or configurations to back them
Security analysis	Before end of April	Report before and after fixes
Legal measures	Before end of April	Proof of filing or justification of its absence, license, agreements
Protection strategy note	Before end of April	One-page note: what is protected, why, how
Technical sheet	Before end of May	Two-page sheet

What you submit

- Map of the elements of value, ranked by criticality
- Inventory of exposure surfaces
- Proof that the measures are implemented: commits, configurations, screenshots
- Security analysis report, before and after fixes
- Proof of filing or justification of its absence, license, any agreements
- Protection strategy note
- Technical sheet
- Demonstration script



B – Open your project to outside contributions

This objective asks you to make your project understandable and usable by developers who have never worked on it, and then to go out and find real contributions.

This objective is distinct from the "take part in technical communities" block in objective 1: there, you contribute to communities that already exist; here, you open your own project and try to bring contributors into it.

Prepare the opening

- A license chosen and justified. The justification has to answer a precise question: what does this choice allow and forbid someone who wants to take the code, modify it, or ship it inside a commercial product? A license named without that analysis does not count.
- A LICENSE file, present and correct.
- A README for a contributor: what the project does, how to install it, how to run the tests, how to propose a change.
- A contribution guide: coding conventions, commit message format, how to submit a proposal, what will and won't be accepted, how fast you aim to reply.
- Scope of the opening: what is open to contributions, what stays under the team's control, and why.
- Issues prepared for newcomers: at least 5 issues open, described so they stand on their own, at least 3 of them doable by someone discovering the project — a documentation fix, an isolated improvement, a missing test.

Go and find contributors

- 8 to 10 relevant venues identified: chat servers, specialist forums, mailing lists, communities around your language or your domain, open-source project directories, student groups at other schools.
- At least 5 calls for contributions published, tailored to each venue, with dated evidence.
- At least 2 follow-ups or reposts, spaced out over time.
- A tracking register: where, when, what message, what audience, what response.

Welcome and integrate contributions

- At least 2 outside contributions received and handled. These count: a proposed code change, a documentation fix, a detailed bug report with a reproduction, a code review by a third party, a translation, an added test.
- The full record of how it was handled: the request, the exchanges, your review, the decision, the merge or the reasoned rejection.



- At least one contribution rejected or substantially modified before merging, with the justification given publicly to the contributor. Accepting everything that arrives is not maintenance.
- Time to first response recorded: how long between the proposal and your first reply. A contributor left without an answer for a week does not come back.
- Contributors credited: a mention in the README, a dedicated file, a public thank-you.

Measure the opening

- Repository traffic: views, clones, stars, where that data is available to you.
- Number of outside people who interacted, in whatever form.
- An accessibility test: at least one person outside the project installs the repository from the contributor documentation alone, and tries to handle an issue. Record how long it took, where they got stuck, and what you fixed in the documentation as a result.

Timeline and deliverables

Item	Due	Deliverable
License and justification	Before end of January	LICENSE file, analysis note on what the license allows
Onboarding documentation	Before end of January	Contributor README, contribution guide, scope of the opening
Newcomer issues	Before end of January	5 issues open, 3 of them suitable for a newcomer
Identifying venues	Before end of February	List of the 8 to 10 venues chosen
Calls for contributions	February to May, 5 calls and 2 follow-ups	Dated register of the calls
Handling contributions	As they arrive	Full record of exchanges and decisions
Accessibility test	Before end of April	Report: who, how long, blockers, fixes applied
Opening review	Before end of May	Traffic record, response times



What you submit

- Link to the public repository, with license, README and contribution guide
- License justification note
- List of newcomer issues
- Dated register of the calls for contributions
- Links to the contributions received and the full exchanges
- Evidence of the rejection or the reasoned modification
- Record of response times
- Accessibility test report

C – Work with technical experts

This objective asks you to go and find an outside, qualified technical eye, to know how to formulate a precise question, to take a substantive critique, and to turn it into decisions.

What is assessed is the approach: how you found the experts, how many you approached, how the exchanges were prepared, and the real effect on the project.

Define what you want challenged

- 2 to 3 technical subjects identified to put forward: an architecture choice, a data model, a technology decision, a security approach, a scaling strategy, an API design.
- For each: what you are not sure about, what makes you hesitate, which options are still open.

Find them and reach out

- Documented search: contributors to projects you use, authors of publications or conference talks on the subject, engineers and architects on professional networks, leads of technical communities, academics, alumni, professionals you met on placement or abroad.
- At least 6 experts approached, with dated evidence of sending.
- At least one follow-up for every approach left unanswered.
- An outreach register: who, what profile, through which channel you found them, when you contacted them, what message, what reply, what outcome.

Prepare and run the exchanges

- A preparation pack sent to the expert at least 48 hours before: the project context in one page, the subject you are putting forward, the options you are considering, the precise questions you are asking. An expert who discovers the subject during the call can only give a general opinion.
- At least 2 actual exchanges, by video or in person.
- A written report within 48 hours: what was said, what was challenged, the recommendations received, any disagreements, the striking quotes.

Turn it into decisions

- At least 2 concrete changes to the project coming out of these exchanges: a component reworked, a technology changed, a security fix, the data model adapted, an architecture choice revised, a tool or practice adopted.
- Traceability: for each change, which exchange it came from, what was changed, when, and the corresponding commit.
- At least one recommendation you did not follow, with the reason. An expert gives an opinion from their context, not yours: being able to say why you are not following it is part of the exercise.

Take stock

- An end-of-year memo: what the exchanges changed in the project, what you learned about your own way of designing things, and what you would do differently in how you reach out.

Timeline and deliverables

Item	Due	Deliverable
Defining the subjects	Before end of December	Note on the 2 to 3 subjects to challenge, with the questions formulated
Outreach	January to April, 6 experts minimum	Dated outreach register, with how you found each one
First exchange	Before end of March	Preparation pack sent, written report
Second exchange	Before end of May	Preparation pack sent, written report
Changes from the exchanges	As you go	Table linking recommendations, decisions and commits
Summary memo	Before end of May	End-of-year memo

What you submit

- Note defining the subjects and the questions asked
- Complete dated outreach register, with how you found each expert
- Sample approach messages
- Preparation packs sent before each exchange
- Reports of the exchanges
- Table linking recommendations, decisions and commits
- Evidence of the recommendation you did not follow, and your reasoning
- Summary memo



D – Measure, test and optimize technical performance

This objective asks you to move out of "it works" and into "it holds up, and here is by how much". It is not about optimizing at random, but about following a complete chain: choose an axis, measure the starting state, diagnose where things block, change something precise, measure again, and observe a gap.

It is the only objective on this track whose result is a figure. That is also what makes it impossible to declare: either the measurement exists and the gap can be read, or nothing is demonstrated.

Choose an axis

- One priority axis chosen from: response or execution time, memory or CPU consumption, behavior under load, client-side loading time, error rate and resilience, running cost, security.
- Justification of the choice: why this axis matters for your project specifically, and what happens if it is not dealt with.
- 2 to 3 measurable indicators defined, each with: its formula, the measurement tool, the conditions under which it is taken.
- A target figure set: what value you are aiming for by the end of the year, and what you base that figure on.

Establish your measurement protocol

- A written protocol: hardware or environment, data set used, scenario run, number of repetitions, when the measurement is taken.
- Reproducibility: the protocol has to let someone else redo the same measurement and get a comparable result.
- Tooling in place for the axis you chose: instrumentation, logging, a profiler, a load testing tool, a dashboard, security analysis.
- Integrated into your development pipeline where possible: measurement run automatically, history kept, an alert on regression.

Measure and diagnose

- A baseline measurement taken and dated on each indicator.
- A diagnosis: where the blockage is. Identified through your tooling (execution profile, slow queries, memory allocation, redundant calls, a vulnerable dependency) and not through intuition.
- An explanatory hypothesis: why this point blocks.
- An estimate of the gain expected if the hypothesis is right.

Change something and measure again

- One precise, isolated change per loop: a query rewritten, caching added, an algorithm changed, a computation moved, a data structure modified, calls batched, a resource lightened, a security fix applied.
- Re-measurement under the exact conditions of the initial protocol.
- The gap observed, expressed in absolute value and in percentage.
- Conclusion: was the hypothesis right, is the gain up to what you expected.
- The trade-off you accept: what the optimization cost in readability, complexity, maintainability, or development time.
- A non-regression check after each loop: the existing tests still pass, and the other indicators on your axis have not been degraded by the optimization. Gaining on response time by doubling memory consumption is not a gain.
- Tracking the indicator over time: a history of the values taken, so you can tell a real gain from measurement noise.

Timeline and deliverables

Item	Due	Deliverable
Axis and indicators chosen	Before end of December	Choice note, indicators with formula and tool, target figure
Measurement protocol	Before end of January	Written, reproducible protocol, tooling in place
Baseline and diagnosis	Before end of February	Dated records, diagnosis report, hypothesis stated
Loop 1	Before end of March	Commit of the change, re-measurement, gap observed
Loop 2	Before end of May	Commit of the change, re-measurement, gap observed
Non-regression	After each loop	Test results, indicator tracking history

What you submit

- Note on the axis chosen and the indicators defined
- Written measurement protocol
- Dated measurement records, initial and final, with screenshots or exports
- Profiling, load testing or analysis reports, depending on your axis
- Commits of the changes made
- Complete before / after table, including the changes that produced no gain
- Non-regression test results
- Indicator tracking history